JavaScript Malware for *Bastards*

You’re a ***Red Teamer***

What do you need *from your malware toolkit* to be successful?

C2 Frameworks as they exist today are (inherently) the result of <u>solutionism</u>

Existing C2 Frameworks: High-Level Flow

Stage 0: Loader (often dropped on disk)

Stage 0: Loader (often dropped on disk)

Stage 1: Retrieves the next stage

Stage 2: Core payload job polling cycle

Detailed Implementation (Example)

1. Produce Position Independent Code (PIC)/shellcode to stage the core C2

Detailed Implementation (Example)

1. Produce Position Independent Code (PIC)/shellcode to stage the core C2
2. Shellcode loader requirement - allocate RW+X memory, either in the current or remote process

Detailed Implementation (Example)

1. Produce Position Independent Code (PIC)/shellcode to stage the core C2
2. Shellcode loader requirement - allocate RW+X memory, either in the current or remote process
3. Trigger shellcode execution - in both local/remote scenarios, the end goal is to have the RIP point to the shellcode address

Detailed Implementation (Example)

1. Produce Position Independent Code (PIC)/shellcode to stage the core C2
2. Shellcode loader requirement - allocate RW+X memory, either in the current or remote process
3. Trigger shellcode execution - in both local/remote scenarios, the end goal is to have the RIP point to the shellcode address
4. Enter the main execution loop:
   a. Poll the C2 address for jobs
   b. Execute the jobs and return results
   c. Apply sleep obfuscation to avoid memory scanning signatures (???)

Why This Design?

The design of having staged payloads with Position Independent Code enables:

- ➔ Prevents static signatures
- ➔ Hampers static reverse engineering (to a degree)
- ➔ Complicates analysis of malware capabilities

What were they trying to solve?

- Whether using BOF/DLL/.NET Assembly
- enable dynamic code execution that allows operators to omit core functionality from the main payload being dropped on disk
- Avoiding:
  - Static signatures
  - Disclosing/burning capabilities

Dynamic Code Execution

SOCKS5 Proxy

is almost always
all you *really* need.

- ➔ Hit internal services
- ➔ Outright avoid most EDR detections
- ➔ Behavioural detections exist are often for *weird* endpoint activity
- ➔ Static detections exist *questionably well* for executable technologies

Malware enables adversaries.

it's rarely the end goal

- There will always be new and old ways to evade static and runtime EDR detections.
- And more so when adversaries operationalize techniques to produce malware that is:
  - harder to signature
  - no anomalous runtime behaviour

produce malware that is:

- harder to signature over N discrete samples
- no anomalous runtime behaviour

(with our dynamic code execution requirements)

What if a simpler, more effective solution existed?

A <u>reliable</u> overlooked approach, simpler … that is far more effective by the very nature of its existence.

Meet JavaScript.

In a (modern) environment, there are <u>far more</u> places than you'd expect of which JavaScript will run

*or you can get to run

Places JavaScript Will Run (for better/worst)

- Browsers
  - A web page!
  - Extensions 😈
  - Service Workers
  - Isolated Web Apps??
  - And many more…
- Electron Applications
  - Getting code execution *depends* on the ASAR's policies
- Standalone
  - Use an existing interpreter (like NodeJS)
  - Single application executable compiler technology (Bun)

Goal: Transform JavaScript source to make it resistant to static analysis and reverse engineering

Let’s take a step back and ask … what makes something **exponentially more frustrating** to reverse engineer?

Classical Obfuscation Techniques

JavaScript’s Implicit Behaviour

Classical Obfuscation Passes

- Control Flow: manipulate the bodies of functions by reordering/inserting bogus/other manipulations of the control flow graph
  - Virtualization
  - Control Flow Flattening
  - Opaque Predicates
- Data: obfuscate the compile time known values through intermediary representations and encoding schemes
  - Data Encoding (different representations)
  - Arithmetic Encoding
  - Literal Encoding
- Functions: re-ordering the function's representation into multiple/inlined/unrelated pieces

https://tigress.wtf/index.html

Classical Obfuscation Techniques

JavaScript's Implicit Behaviour

JavaScript has a lot of “gotchas”...

Proficient and Experienced <u>developers often find themselves being caught off guard</u> by JavaScript’s nuances.

What if we **systematically categorized** these ‘gotchas’ for fun *(and no profit)*

JavaScript runtime behaviour we can systematically abuse for obfuscation while preserving strict semantic equivalence..

So let’s do it…

Here’s almost every **semantic invariant** I could pull from the JS scoped to E5

Completion Precedence in `try`/`finally`

- A completion produced by `finally` supersedes an earlier completion from `try` or `catch`.
- This includes both normal and abrupt completions (`return`, `throw`).
- Static reading of `try` block exit often becomes misleading.

```
Proof:

function f(){ try { return 1; } finally { return 2; } }
function g(){ try { return 1; } finally { throw new Error("boom"); } }
console.log(f());
try { console.log(g()); } catch (e) { console.log(e.message); }

Observed:

- 2
- boom
```

Direct vs Indirect `eval` Scope Mode

- Direct `eval(...)` executes in current lexical environment.
- Indirect `eval` (e.g. `(0, eval)(...)`) executes in global environment.
- Gives a scope-selection mux without explicit closures/proxy environments.

```
Proof:

globalThis.__es5_eval_probe = 1;
function scopeProbe() {
  var __es5_eval_probe = 2;
  return [eval("__es5_eval_probe"), (0, eval)("__es5_eval_probe")];
}
console.log(scopeProbe());

Observed:

- [2, 1]
```

Declaration Injection Behavior

- Direct eval can inject local `var` bindings into current function scope.
- Indirect eval can create globals.
- Hidden symbol introduction and delayed binding creation are powerful for control/data indirection.

```
Proof:


function localInject(){ eval("var local_injected = 9"); return local_injected; }
(function(){ (0, eval)("var global_injected = 11"); })();
console.log(localInject(), globalThis.global_injected);
delete globalThis.global_injected;

Observed:

- 9 11
```

Sloppy `this` Default Binding + Primitive Boxing

- In non-strict plain calls, `this` defaults to global object.
- `Function.prototype.call/apply` with primitive receiver boxes to wrapper object.
- Introduces call-form-dependent behavior/hidden receiver coercion pathways

```
Proof:

function thisProbe(){ return this === globalThis; }
function boxedProbe(){ return [typeof this, this instanceof Number, this.valueOf()];
}
console.log(thisProbe());
console.log(boxedProbe.call(3));

Observed:

- true
- ["object", true, 3]
```

Sloppy vs Strict `arguments` Mapping Split

- Non-strict functions may map parameter identifiers to `arguments` indices.
- Strict functions do not.
- A deterministic semantic branch on strictness domain with identical syntax shape.

```
Proof:

function sloppy(a){ a = 7; return arguments[0]; }
function strictMode(a){ "use strict"; a = 7; return arguments[0]; }
console.log(sloppy(1), strictMode(1));

Observed:

- 7 1
```

`with` Object Environment Rebinding

- `with (obj) { ... }` changes identifier resolution to include object properties first.
- Runtime name-resolution rewriting without AST mutation.

```
Proof:

var wx = 1;
const wo = { wx: 2 };
with (wo) { wx = 3; }
console.log(wx, wo.wx);

Observed:

- 1 3 (assignment resolved to object property, not outer binding)
```

ToPrimitive Conversion Pivot (`valueOf` then `toString`)

- Arithmetic/concatenation operations trigger object-to-primitive conversion protocol.
- Side effects can be hidden in coercion hooks rather than explicit calls.

```
Proof:

const trace = [];
const obj = {
  valueOf(){ trace.push("valueOf"); return 5; },
  toString(){ trace.push("toString"); return "9"; },
};
console.log(obj + 1, trace);

Observed:

- 6
- ["valueOf"]
```

Accessor Descriptor Execution (Lazy Materialization)

- Getter invocation is deferred until property read.
- Property can still appear enumerable while value is computed dynamically.
- Supports decode-on-read and read-count dependent behaviors.

```
Proof:

let hits = 0;
const box = {};
Object.defineProperty(box, "x", { enumerable: true, get(){ hits++; return 42; } });
console.log(box.x, hits, Object.keys(box));

Observed:

- 42 1 ["x"]
```

Descriptor Invariants (configurable: false)

- Non-configurable properties cannot be redefined incompatibly.
- Deletion of non-configurable own property fails (`false` in non-strict mode).
- Strong anti-tamper anchor at object-shape level.

```
Proof:

const b = {};
Object.defineProperty(b, "x", { value: 1, writable: false, configurable: false });
try { Object.defineProperty(b, "x", { value: 2 }); } catch (e) { console.log(e.name);
}
const d = {};
Object.defineProperty(d, "x", { value: 1, configurable: false });
console.log(b.x, delete d.x, d.x);

Observed:

- TypeError
- 1 false 1
```

Integrity Level Transitions (`freeze`)

- `Object.freeze` prevents extension and locks own properties.
- One-way state marker for staged obfuscation lifecycles.

```
Proof:

const frozen = { a: 1 };
Object.freeze(frozen);
frozen.a = 9;
delete frozen.a;
console.log(frozen.a, Object.isFrozen(frozen), "a" in frozen);

Observed:

- 1 true true
```

Array Exotic `length` Semantics

- Reducing `length` truncates indexed elements above new length.
- Bulk structural mutation with a single write; good for compact state resets.

```
Proof:

const a = [1, 2, 3, 4];
a.length = 2;
console.log(a, 2 in a, 3 in a);


Observed:

- [1,2] false false
```

`for...in` Includes Enumerable Inherited Properties

- Enumeration walks enumerable own + inherited properties.
- Prototype chain can carry hidden control state into loops.

```
Proof:

const p = { a: 1 };
const o = Object.create(p);
o.b = 2;
const keys = [];
for (const k in o) keys.push(k);
console.log(keys.sort());

Observed:

- ["a","b"]
```

JSON `toJSON` + Replacer Pipeline

- `toJSON` runs before replacer callback.
- Two-stage transformation channel during serialization.

```
Proof:

const payload = { a: 1, toJSON(){ return { z: 9 }; } };
console.log(JSON.stringify(payload, (k, v) => (k === "z" ? v + 1 : v)));

Observed:

- {"z":10}
```

RegExp Replace Callback Protocol

- Callback receives captures, index, and source string.
- Rich callback signature can encode structured metadata flow.

```
Proof:

const trace = [];
const out = "ab".replace(/(a)(b)/, (_, a, b, idx, s) => {
  trace.push([a, b, idx, s]);
  return b + a;
});
console.log(out, trace);


Observed:

- ba
- [["a","b",0,"ab"]]
```

`defineProperty` Default Attributes

- If omitted, descriptor attributes default to `false` (`writable`, `enumerable`, `configurable`).
- Hidden constants can be created that are non-enumerable and hard to patch.

```
Proof:

const o = {};
Object.defineProperty(o, "x", { value: 1 });
o.x = 2;
console.log(o.x, Object.keys(o).length, delete o.x);



Observed:

- 1 0 false
```

Data/Accessor Descriptor Conflict

- A descriptor cannot simultaneously define data and accessor fields.
- Descriptor construction can be used as integrity check; malformed rewriters leak via TypeError.

```
Proof:

try {
  Object.defineProperty({}, "x", { value: 1, get() { return 2; } });
} catch (e) {
  console.log(e.name);
}

Observed:

- TypeError
```

`preventExtensions` Structural Gate

- New own properties cannot be added once object is non-extensible.
- Locks runtime object shape and blocks late injection.

```
Proof:

const o = { a: 1 };
Object.preventExtensions(o);
o.b = 2;
console.log("b" in o, Object.isExtensible(o));


Observed:

- false false
```

`seal` vs `freeze` Distinction

- Sealed objects block add/delete but may still allow writes on writable fields.
- Frozen objects additionally block writes
- Enables staged lockdown strategies

```
Proof:

const o = { a: 1 };
Object.seal(o);
o.a = 2;
delete o.a;
console.log(o.a, Object.isSealed(o), Object.isFrozen(o));

Observed:

- 2 true false
```

Sparse Array Hole Iteration

- Hole indices are skipped by callbacks in `map` and `forEach`.
- Presence/absence of holes can encode state without explicit sentinels.

```
Proof:

const a = [, 1, , 2];
const m = [];
const f = [];
a.map((v, i) => { m.push(i + ":" + v); return v; });
a.forEach((v, i) => f.push(i + ":" + v));
console.log(m, f, a.length);

Observed:

- ["1:1","3:2"] ["1:1","3:2"] 4
```

`in` Operator vs `undefined` Value

- `in` checks property existence, not value definedness.
- Distinguishes "present but undefined" from "absent".

```
Proof:

const o = { x: undefined };
console.log("x" in o, o.hasOwnProperty("x"), o.x === undefined);

Observed:

- true true true
```

Dynamic Prototype Lookup

- Property resolution follows current prototype chain state at access time.
- Prototype mutation can rewire behavior for existing instances.

```
Proof:

const base = { a: 1 };
const o = Object.create(base);
console.log(o.a);
base.a = 2;
console.log(o.a);
delete base.a;
console.log(o.a);

Observed:

- 1
- 2
- undefined
```

`instanceof` Prototype Pivot

- `instanceof` uses constructor `.prototype` identity at check time.
- Post-construction prototype replacement changes membership outcomes.

```
Proof:

function F() {}
const x = new F();
const old = F.prototype;
F.prototype = {};
console.log(x instanceof F);
F.prototype = old;
console.log(x instanceof F);

Observed:

- false
- true
```

`Function` Constructor Global Scope Evaluation

- `Function("...")` body executes in global scope, not local lexical scope.
- Hidden global read/eval channels can bypass local shadowing assumptions.

```
Proof:

globalThis.__fn_scope = 1;
function g() { var __fn_scope = 2; return Function("return __fn_scope")(); }
console.log(g());
delete globalThis.__fn_scope;

Observed:

- 1
```

Short-Circuit Sequencing Gate

- `&&` and `||` evaluate left operand first and conditionally evaluate right operand.
- Cheap execution gating with value propagation and side-effect timing control.

```
Proof:

const t = [];
function A() { t.push("A"); return 0; }
function B() { t.push("B"); return 1; }
A() && B();
A() || B();
console.log(t);

Observed:

- ["A","A","B"]
```

`catch` Binding Scope Isolation

- `catch (e)` binding is block-scoped to catch clause.
- Captured value can be exported manually (e.g., via `var` assignment).
- Scoped exception payloads can be selectively leaked.

```
Proof:

let errName = "none";
try { throw 5; } catch (e) { var fromCatch = e; }
try { e; } catch (x) { errName = x.name; }
console.log(fromCatch, errName);


Observed:

- 5 ReferenceError
```

Getter Abrupt Completion Surface

- Getter execution can throw during ordinary property read expressions.
- Passive-looking reads can become abrupt control transfer sites.

```
Proof:

const o = { get x() { throw new Error("boom-x"); } };
try { console.log(o.x + 1); } catch (e) { console.log(e.message); }


Observed:

- boom-x
```